

L'ingegneria del software non riguarda la semplice stesura di codice, ma l'applicazione di un approccio sistematico e quantificabile per creare e mantenere sistemi complessi. A differenza della programmazione individuale, richiede un lavoro di squadra e un dialogo continuo con i committenti, chiamati stakeholder.

1. Il Ciclo di Vita del Software

La realizzazione di un prodotto attraversa diverse fasi che, sebbene non rigidamente sequenziali, sono indispensabili per strutturare il lavoro.

Fase	Descrizione e Obiettivo
Studio di Fattibilità	Analisi preliminare di costi, benefici e risorse necessarie per capire se procedere.
Analisi dei Requisiti	Identificazione formale delle specifiche. I requisiti si dividono in funzionali (relazione input-output) e non funzionali (qualità generali).
Progettazione	Definizione dell'architettura generale e scomposizione in moduli di basso livello. Viene definita l'interfaccia di ogni modulo, ma non i dettagli interni.
Codifica e Test	Implementazione del codice sorgente e test individuale dei singoli moduli.
Integrazione	Assemblaggio dei moduli testati e verifica finale dell'intero sistema.
Consegna e Manutenzione	Rilascio al cliente e successiva manutenzione. Questa è la fase più lunga e fondamentale di tutto il ciclo.

2. Le Qualità del Software

Il prodotto finale è un sistema duttile, che deve garantire precise qualità misurabili, divise tra esterne (visibili all'utente) e interne (riguardanti gli sviluppatori).

- **Correttezza e Affidabilità:** Un software è corretto se rispetta in modo assoluto le specifiche stabilite. L'affidabilità è invece la probabilità che mantenga il comportamento atteso nel tempo.
- **Robustezza:** La capacità di gestire in modo accettabile situazioni non previste dalle specifiche iniziali.
- **Prestazioni:** Riguardano l'efficienza e l'uso ottimale delle risorse hardware, valutabili tramite misurazioni dirette, analisi o simulazioni.
- **Manutenibilità ed Evolvibilità:** Quanto il sistema è aperto a interventi successivi (correttivi o perfettivi) e capace di evolvere per adattarsi a nuovi scenari.
- **Riusabilità e Interoperabilità:** La possibilità di riutilizzare il software per altri progetti (es. librerie, design pattern) e la sua capacità di cooperare con altri sistemi tramite interfacce standard.

3. Principi Fondamentali dell'Ingegneria del Software

Per garantire queste qualità, il processo di sviluppo si basa su criteri metodologici rigorosi.

- **Rigorosità e Formalità:** La precisione nella documentazione e nello sviluppo aiuta a rispettare le tempistiche.

- **Separazione degli Interessi:** Affrontare i problemi singolarmente dividendo il lavoro per tempistiche, qualità o responsabilità.
- **Modularità:** Scomporre un sistema complesso in parti semplici (approccio top-down o bottom-up). È vitale mantenere un'alta coesione interna e un basso accoppiamento tra i moduli. Questo permette di isolare comportamenti specifici (come interazioni fisiche complesse in un motore grafico) trattandoli come "black box" e ignorando la loro struttura interna durante la visione d'insieme.
- **Astrazione e Incrementabilità:** Ignorare i dettagli di basso livello per concentrarsi sugli aspetti chiave e procedere per approssimazioni successive, rilasciando prototipi e raccogliendo feedback.

4. Modelli e Metodologie di Produzione

L'approccio grezzo "Code and Fix" (scrivi il codice e correggilo all'infinito) è diventato insostenibile con l'aumento della complessità delle applicazioni. Sono nati così modelli strutturati:

Modelli Tradizionali

- **Modello Waterfall (A Cascata):** Fasi rigorosamente sequenziali dove l'output di una è l'input dell'altra. È rigido, non prevede parallelismo e gestisce male i cambiamenti in corso d'opera.
- **Modello a Spirale:** Basato sulla valutazione progressiva di costi e rischi attraverso 4 quadranti, procedendo per incrementi.

Metodologie Strutturate (UP/RUP)

- Il RUP divide lo sviluppo in quattro macro-fasi: Concezione, Elaborazione, Costruzione e Transizione.
- È un approccio iterativo guidato dai casi d'uso, fortemente centrato sull'architettura e basato sulla programmazione orientata agli oggetti e UML.

Metodologie Agili (XP e Scrum) Nate per rispondere rapidamente ai cambiamenti del mercato, si basano su sviluppo iterativo, rilasci continui e minor peso alla documentazione formale a favore del codice.

- **Extreme Programming (XP):** Estremizza lo sviluppo incrementale con rilasci frequenti. Utilizza pratiche come il *refactoring* continuo, il *test-first development* e la programmazione in coppia.
- **Scrum:** Ottimizzato per lo sviluppo iterativo. I requisiti sono raccolti in un *product backlog* (to-do list). Il lavoro è diviso in cicli detti *sprint* (di 2-4 settimane), mediati da uno *Scrum Master* che facilita il lavoro del team senza esserne il capo. Le metodologie agili sono eccellenti per team piccoli, ma difficili da scalare su sistemi software mastodontici.